

Taxi-v3 Reinforcement Learning Assignment

Krishhiv Mehra - 1020231341

November 20, 2025

1 Introduction

This project investigates different approaches to solving the classic **Taxi-v3** environment from OpenAI Gymnasium. The task is to control a taxi in a 5×5 grid world: the taxi must navigate to a passenger location, pick up the passenger, move to the destination and drop them off. The environment provides:

- a small negative reward (-1) at every time step,
- a large positive reward for a successful pick-up and drop-off each (approximately $+10$),
- a heavy penalty for an illegal drop-off or pick-up (approximately -10).

The goal of the agent is to learn a policy that maximises the long-term discounted return, i.e., complete trips quickly while avoiding penalties.

The repository contains a progression of models:

1. Simple baselines (random agent and a Dijkstra shortest-path planner),
2. A tabular Q-learning agent,
3. An optimised version of Q-learning with different exploration schedules,
4. A Double Q-learning agent that addresses overestimation bias.

All models are evaluated in terms of average total reward, the number of steps per episode and the range of returns obtained.

2 Baseline Models

2.1 Random Agent

The random agent provides a “learning nothing” baseline. In every state it selects an action uniformly at random from the six available actions (move north/south/east/west, pick up, drop off). It has:

- no memory of the past,
- no learning, and
- no knowledge of the environment structure.

As expected, this agent wanders around the grid, frequently attempts illegal pickups or drop-offs and usually receives a large negative total reward. This baseline is useful as a lower bound: any learning algorithm should perform significantly better than the random policy.

2.2 Dijkstra Shortest-Path Planner

The Dijkstra-based planner is a baseline model that assumes access to the grid layout and legal moves. The grid is represented as a weighted graph where:

- each grid cell is a node,
- edges connect adjacent cells with cost 1.

For each episode, the planner computes:

1. the shortest path from the taxi’s current position to the passenger (for pick-up),
2. the shortest path from the passenger’s location to the destination (for drop-off).

These paths are converted into environment actions (directional moves followed by a **pickup** or **dropoff**). Because it plans in the true environment model, this agent is close to optimal in path length and avoids most illegal actions.

Dijkstra Outputs

Over 200 evaluation episodes using the Dijkstra planner, the following statistics were obtained:

- **Average reward:** 8.14
- **Average steps:** 12.86
- **Maximum reward:** 15
- **Minimum reward:** 3

Figure 1 shows the per-episode total reward. Although there is episode-to-episode variability due to different starting configurations for passenger and destination, the planner consistently achieves high positive returns and short trajectories.

Dijkstra therefore acts as an approximate “upper bound” on what a perfectly modelled agent can achieve in this environment.

3 Q-learning Model

3.1 Method

Taxi-v3 has a discrete state space of 500 states (taxi row, taxi column, passenger location index and destination index) and 6 discrete actions. A tabular Q-learning agent maintains a Q-table $Q(s, a)$ for all state-action pairs and updates it using the standard rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where s and a are the current state and action, r is the reward, s' is the next state, α is the learning rate and γ is the discount factor.

Action selection uses an ε -greedy policy: with probability ε the agent chooses a random action; otherwise it chooses the greedy action $\arg \max_a Q(s, a)$. During training ε decays over time to reduce exploration as learning progresses.

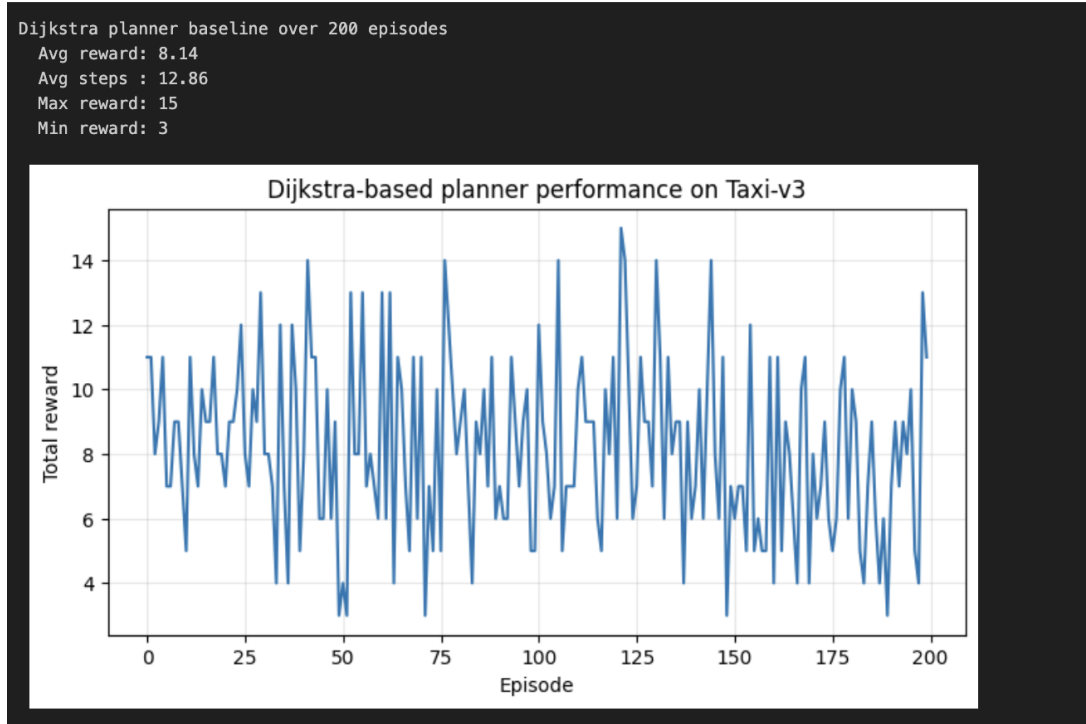


Figure 1: Dijkstra-based planner performance on Taxi-v3 over 200 episodes.

3.2 Outputs for Q-learning

Two Q-learning models were trained that differ only in the decay rate of ϵ :

- **Model A:** $\epsilon_{\text{decay}} = 0.995$
- **Model B:** $\epsilon_{\text{decay}} = 0.99$

After training, both models were evaluated in a *greedy* mode ($\epsilon \approx 0$) over 3000 episodes. The results are:

Model A (with `eps_decay = 0.995`)

- Average reward: 7.878
- Average steps: 13.122
- Maximum reward: 15
- Minimum reward: 3
- Maximum steps: 18
- Minimum steps: 6

Model B (with `eps_decay = 0.99`)

- Average reward: 7.873
- Average steps: 13.127
- Maximum reward: 15

- Minimum reward: 3
- Maximum steps: 18
- Minimum steps: 6

The two ε schedules lead to almost identical final performance: in both cases the Q-learning agent learns a near-optimal policy that consistently picks up and drops off passengers with relatively short paths and a high average return.

4 Double Q-learning Model

4.1 Method

Standard Q-learning can suffer from *overestimation bias* because the $\max_{a'} Q(s', a')$ term tends to pick overoptimistic estimates when Q-values are noisy. Double Q-learning addresses this by maintaining two independent Q-tables, Q^A and Q^B , and decoupling action selection from value evaluation.

At each update step:

- With probability 0.5 we update Q^A ; otherwise we update Q^B .
- When updating Q^A :

$$a^* = \arg \max_a Q^A(s', a), \quad Q^A(s, a) \leftarrow Q^A(s, a) + \alpha [r + \gamma Q^B(s', a^*) - Q^A(s, a)].$$

- When updating Q^B , the roles of A and B are swapped.

For action selection during interaction, the agent uses the combined value $Q^{\text{combined}}(s, a) = Q^A(s, a) + Q^B(s, a)$ inside the same ε -greedy strategy.

4.2 Outputs for Double Q-learning

After training Double Q-learning for a sufficiently large number of episodes (so that under-training is not an issue), the greedy policy was evaluated over 3000 episodes. The performance statistics are:

- Average reward: 7.941
- Average steps: 13.059
- Maximum reward: 15
- Minimum reward: 3
- Maximum steps: 18
- Minimum steps: 6

Double Q-learning achieves slightly higher average rewards and slightly fewer steps than the single Q-learning baselines, indicating that with enough training it can match or marginally improve on vanilla Q-learning in this small tabular setting.

Model	Avg. reward	Avg. steps	Notes
Random agent	strongly negative	very high	No learning or planning
Dijkstra planner	8.14	12.86	Near-optimal, model-based
Q-learning (A)	7.878	13.122	$\epsilon_{\text{decay}} = 0.995$
Q-learning (B)	7.873	13.127	$\epsilon_{\text{decay}} = 0.99$
Double Q-learning	7.941	13.059	Two Q-tables, reduced bias

Table 1: Summary of performance across models (greedy evaluation).

5 Takeaways and Model Comparison

5.1 Quantitative comparison

Table 1 summarises the key metrics for the main models.

The random agent performs extremely poorly and mainly serves as a sanity check. All three learnt RL policies (Q-learning A/B and Double Q-learning) are vastly better than random and achieve rewards close to the Dijkstra upper baseline.

The Dijkstra planner still has a slight edge in terms of both average reward and average number of steps, however, it may fluctuate, but it still shows a slight edge which expected - it has exact knowledge of the environment dynamics and explicitly plans shortest paths. The RL agents, however, approach this performance *purely from trial-and-error experience*, without being given any model of the environment.

5.2 Optimal model choice

From a *planning* perspective, Dijkstra is the most optimal method in this project: it obtains the highest average reward and the fewest steps, and it is guaranteed to find shortest paths under the modelling assumptions.

From a *reinforcement learning* perspective, Double Q-learning can be considered the strongest model:

- It slightly outperforms the single Q-learning baselines in both average reward and steps.
- It addresses the theoretical overestimation issue in standard Q-learning by decoupling action selection and evaluation.
- It demonstrates that with sufficient training data, model-free RL can learn a policy that is competitive with a handcrafted planner.

Q-learning itself remains very competitive and significantly simpler to implement. On this small discrete problem, the advantage of Double Q-learning is modest; its main value is conceptual and becomes more important in larger and noisier environments.

5.3 Conceptual lessons

The project highlights several key ideas:

- **Baselines matter:** comparing against both a random agent and a near-optimal planner helps to contextualise RL performance.
- **Exploration and hyperparameters:** the choice of learning rate, discount factor and ϵ -schedule critically affects how quickly and how well Q-learning converges.

- **Model-based vs model-free:** Dijkstra uses full knowledge of the environment, whereas Q-learning and Double Q-learning rely only on sampled transitions and rewards.

Overall, the experiments show that tabular RL methods can learn strong policies for Taxi-v3 and approach the performance of an explicit shortest-path planner, while also providing a good demonstration of the effects of exploration, learning dynamics and overestimation bias.